Генетические алгоритмы

Проблема

В разделе про ИИ врагов мы вручную настраивали параметры для трёх типов врагов:

Параметр	Стражник	Берсерк	Лучник
Скорость	60	120	50
Радиус обнаружения	150	300	250
Дистанция атаки	40	50	200
Урон	10	25	15

Ладно, три типа - нормально, руками подобрали. А если типов 50? А если у каждого врага 10 параметров? А если вы хотите, чтобы враги были **сбалансированными** - не слишком сильными, не слишком слабыми, а создающими интересный бой?

Наивный подход: генерируем случайные наборы параметров, тестируем каждый, выбираем лучший.

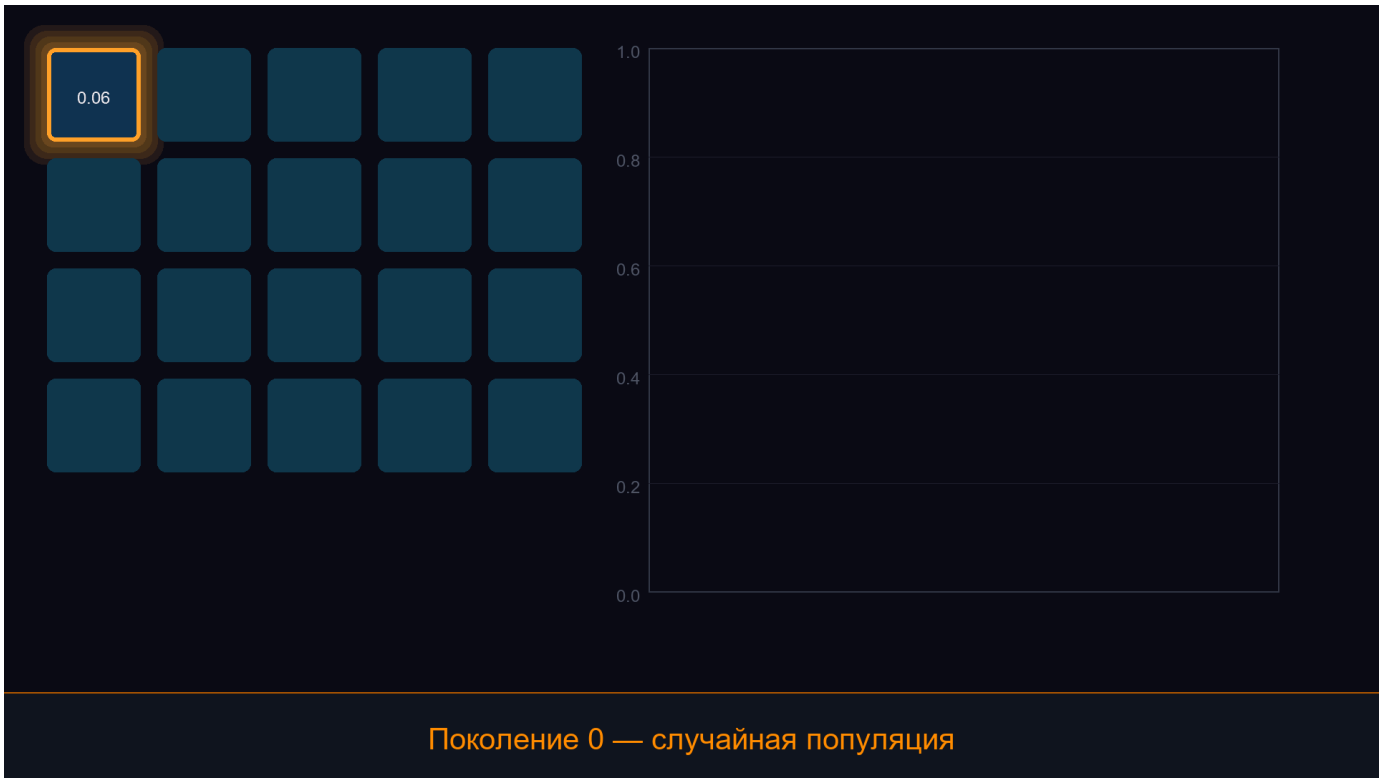
Python C#

```python
import random

def random_enemy():
    return {
        "speed": random.uniform(30, 200),
        "hp": random.uniform(20, 200),
        "damage": random.uniform(5, 50),
        "detection": random.uniform(50, 400),
    }

def test_enemy(params):
    """Simulate a fight, return score: higher = more interesting enemy."""
    # ... some simulation logic ...
    return random.random()  # placeholder

```

```
16
17 best_score = 0
18 best_enemy = None
19 for _ in range(10000):
20     enemy = random_enemy()
21     score = test_enemy(enemy)
22     if score > best_score:
23         best_score = score
           best_enemy = enemy
```

Получаем кучу независимых попыток. Алгоритм не **учится** на предыдущих результатах. Нашёл врага с хорошей скоростью и врага с хорошим уроном - но не может скомбинировать их достоинства или найти нечто среднее. И вот 10 000 попыток вслепую.

А что, если каждая новая попытка **помнила** лучшие результаты и комбинировала их сильные стороны?



Идея: эволюция вместо перебора

Метафора из биологии. В природе выживают наиболее приспособленные. Они скрещиваются, их потомки наследуют удачные комбинации генов. Случайные мутации иногда дают признаки получше - и они тоже закрепляются.

Генетический алгоритм (ГА), по сути, про это:

- **Популяция** - набор кандидатов (например, 20 вариантов врагов).
- **Хромосома** (individual) - один кандидат. Список чисел: `[speed, hp, damage, detection]` .

- **Ген** - один параметр внутри хромосомы.
- **Фитнес-функция** - оценка качества кандидата. В нашем случае, к примеру - "Насколько интересен бой с этим врагом?". Скажем, среднее количество ходов до конца битвы? Соотношение побед/поражений в схватках с игроком? Друг с другом?

Цикл: создать популяцию -> оценить приспособленность каждого -> отобрать лучших -> скрестить -> мутировать -> повторить. Через десятки поколений популяция "сходится" к хорошему решению.

Хромосома и фитнес-функция

Хромосома - это, грубо говоря, просто список чисел. Для врага: `[speed, hp, damage, detection_radius]` . Для оружия: `[damage, fire_rate, reload_time]` . Для уровня: сетка 0/1.

Фитнес-функция - самая важная часть. Она определяет, что значит "хороший" кандидат. Для врага это может быть результат симуляции боя: враг, который убивает игрока слишком быстро - плохой (неинтересно), который умирает за секунду - тоже плохой. Хороший - тот, с которым бой длится 10–20 секунд.

Python C#

```python
import random

class Individual:
    def __init__(self, genes=None):
        if genes:
            self.genes = list(genes)
        else:
            # random initialization
            self.genes = [
                random.uniform(30, 200),   # speed
                random.uniform(20, 200),   # hp
                random.uniform(5, 50),     # damage
                random.uniform(50, 400),   # detection radius
            ]
        self.fitness = 0.0

    def evaluate(self):
        speed, hp, damage, detection = self.genes
        # example: balanced enemy = moderate challenge
        fight_duration = hp / max(damage * 0.5, 1)
        aggression = speed * detection / 10000
        # ideal fight ~15 sec (deviation × 5), aggression ~3 (deviation × 
        # example: fight_duration=20, aggression=1 → 100 - 5*5 - 2*10 = 55
        # example: fight_duration=15, aggression=3 → 100 - 0 - 0 = 100 (per
```

```
        self.fitness = max(0, 100 - abs(fight_duration - 15) * 5
                                  - abs(aggression - 3) * 10)
```

Фитнес-функция - та же идея, что **оценочная функция** в минимаксе. Там мы оцениваем позицию на доске, здесь - качество кандидата. Разный контекст, один принцип: присвоить число, чтобы алгоритм мог сравнивать.

Отбор (Selection)

Из популяции нужно выбрать родителей для следующего поколения. Идея: лучшие кандидаты должны размножаться чаще, но и "средние" должны иметь шанс - иначе популяция застрянет в локальном оптимуме.

Простейший метод - **турнирный отбор**: берём K случайных кандидатов, побеждает тот, у кого фитнес выше. K = 3 - хороший баланс между давлением отбора и разнообразием.

Рулеточный отбор (roulette wheel selection) - это в точности **взвешенный случайный выбор** из раздела про случайность. Вес каждого кандидата = его фитнес. Чем выше фитнес, тем больше шанс быть выбранным.

Скрещивание (Crossover)

Два родителя -> два потомка. Берём гены частично от одного, частично от другого. Простейший вариант - **одноточечное скрещивание**: выбираем случайную точку разреза, потомок получает гены первого родителя до точки и второго - после.

Python C#

```python
def crossover(parent_a, parent_b):
    point = random.randint(1, len(parent_a.genes) - 1)
    child1_genes = parent_a.genes[:point] + parent_b.genes[point:]
    child2_genes = parent_b.genes[:point] + parent_a.genes[point:]
    return Individual(child1_genes), Individual(child2_genes)
```

Родитель A - быстрый хилый. Родитель B - медленный живучий. Потомок может унаследовать скорость A и здоровье B - комбинацию, которую случайный перебор искал бы очень долго.

Мутация (Mutation)

Скрещивание комбинирует существующие гены, но не создаёт **новых** значений. Мутация добавляет случайные отклонения с вероятностью `mutation_rate`. Без мутации популяция быстро теряет разнообразие и застревает.

Python C#

```python
def mutate(individual, mutation_rate=0.1, strength=0.2):
    for i in range(len(individual.genes)):
        if random.random() < mutation_rate:
            delta = individual.genes[i] * random.uniform(-strength, strengt
            individual.genes[i] += delta
```

`strength = 0.2` - мутация отклоняет значение на +-20%. Это достаточно, чтобы исследовать окрестности, но не настолько, чтобы разрушить хорошую хромосому. Подробнее о генераторе случайных чисел и seed'ах - в соответствующем разделе.

Типичный вывод:

```
Gen 0:  best=32.4  avg=8.7
Gen 10: best=71.2  avg=45.3
Gen 30: best=89.8  avg=72.1
Gen 49: best=94.6  avg=85.3
```

За 50 поколений фитнес вырос с ~30 до ~95. Ни одна конкретная комбинация не была запрограммирована - алгоритм **нашёл** её сам.

Обратите внимание на **элитизм** - лучший кандидат копируется в следующее поколение без изменений. Без этого скрещивание и мутация могут "потерять" удачное решение.

Весь алгоритм - ~40 строк кода. Но качество результата зависит от **фитнес-функции**. Если фитнес неправильно оценивает "хорошего" врага, ГА оптимизирует не то, что нужно. Мусор на входе - мусор на выходе. Довольно часто встречающийся принцип. Например, в нейронках.

Одни параметры - разные задачи

Тот же код, разные хромосомы и фитнес-функции - и ГА решает совершенно разные задачи:

Задача	Хромосома	Фитнес-функция
Параметры врагов	`[speed, hp, dmg, radius]`	Результат симуляции боя
Раскладка уровня	Сетка 0/1	Проходимость + число комнат
Баланс оружия	`[damage, firerate, reload]`	Винрейт в симуляции
Цветовая палитра	`[r,g,b, r,g,b, ...]`	Контрастность + гармония

Это знакомый паттерн: одна и та же архитектура, разные данные. Как таблица параметров врагов из ИИ врагов - один автомат, разные числа.

Когда НЕ нужен ГА

ГА - не серебряная пуля. Если есть способ лучше - используйте его:

- **Маленькое пространство поиска** - переберите все варианты.
- **Есть градиент** - используйте градиентный спуск (быстрее и точнее).
- **Проблема имеет структуру** - A*, минимакс, конкретный алгоритм.
- **Нужен оптимальный ответ** - ГА находит *хорошее* решение, не *лучшее*.

ГА хорош, когда: пространство огромное, структура неизвестна, градиент не вычислить, а "достаточно хорошее" решение устраивает.

Эволюционные вычисления

Генетический алгоритм - частный случай целого семейства подходов:

- **Эволюционные стратегии (ES)** - похоже на ГА, но без скрещивания. Только мутация. Работает лучше для непрерывных пространств (float-параметры).
- **Генетическое программирование** - эволюция не чисел, а **кода**. Хромосома - дерево операций. Можно "вырастить" формулу фитнеса или правило поведения.
- **NEAT** (NeuroEvolution of Augmenting Topologies) - эволюция нейросетей. Не только веса, но и **структура** сети (как нейроны соединены) эволюционирует.
- **Роевой интеллект** - похожая идея, но вместо генетики - социальное поведение. Частицы "летают" по пространству решений и обмениваются информацией. Boids из раздела про управляющие силы - визуально похожий принцип.
- **Многокритериальная оптимизация** - что, если фитнес-функций несколько? "Враг должен быть и интересным, и не слишком сильным". Парето-фронт - множество решений, где нельзя улучшить одно, не ухудшив другое.

Что	Где подробнее
Параметры врагов, которые можно эволюционировать	Простой ИИ врагов
Случайность, seed, взвешенный выбор	Случайность и вероятность
Дерево поиска вместо популяции	Минимакс
Генерация уровней как фитнес-ландшафт	Процедурная генерация
Проверка проходимости уровня (flood fill)	Уровни и поиск путей
Нейросеть как выученная фитнес-функция	Нейросети для игр

Powered by Wiki.js